

BÀI TẬP 9 – TRIGGER

Hệ quản trị cơ sở dữ liệu Oracle – PL/SQL

TÓM TẮT CÚ PHÁP TRIGGER – ORACLE PL/SQL

1. Cấu trúc tổng quát của Trigger

CREATE OR REPLACE TRIGGER

```
CREATE OR REPLACE TRIGGER ten_trigger
BEFORE | AFTER    INSERT | UPDATE [OF col] | DELETE
ON ten_bang
[FOR EACH ROW]    -- Trigger mức dòng (Row-level)
[WHEN (dieu_kien)]
DECLARE
    -- Khai báo biến (nếu cần)
BEGIN
    -- Thân trigger
    -- :NEW.col → giá trị mới (INSERT/UPDATE)
    -- :OLD.col → giá trị cũ (DELETE/UPDATE)
END ten_trigger;
/
```

2. So sánh Oracle vs SQL Server

Điểm khác biệt quan trọng

-- SQL Server		Oracle PL/SQL
-- -----		-----
-- FOR INSERT		AFTER INSERT / BEFORE INSERT
-- INSERTED.col		:NEW.col
-- DELETED.col		:OLD.col
-- RAISERROR('msg',16,1)		RAISE_APPLICATION_ERROR(-20001, 'msg')
-- ROLLBACK TRANSACTION		(tự động rollback khi có lỗi)
-- IF UPDATE(col)		UPDATING('col') (dùng trong trigger)
-- Không có FOR EACH ROW		Mặc định là statement-level (không FOR EACH ROW)

3. BEFORE Trigger – Ngăn DML khi có lỗi

Dùng BEFORE để kiểm tra và hủy thao tác

```
CREATE OR REPLACE TRIGGER trg_vi_du
BEFORE INSERT ON ten_bang
FOR EACH ROW
DECLARE
    v_dem NUMBER := 0;
BEGIN
    SELECT COUNT(*) INTO v_dem FROM bang_khac WHERE ma = :NEW.ma;
    IF v_dem = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Ma khong ton tai');
        -- Oracle tự động rollback câu lệnh INSERT gây ra trigger
    END IF;
END trg_vi_du;
/
```

4. AFTER Trigger – Cập nhật bảng khác

Dùng AFTER để cập nhật dữ liệu sau khi DML xảy ra

```
CREATE OR REPLACE TRIGGER trg_sau_insert
AFTER INSERT ON hoadon
FOR EACH ROW
BEGIN
    -- Giảm số lượng tồn kho
    UPDATE hang
    SET soluong = soluong - :NEW.soluongban
    WHERE mahang = :NEW.mahang;
END trg_sau_insert;
/
```

5. Compound Trigger – Kiểm soát số dòng bị ảnh hưởng

Dùng Compound Trigger khi cần đếm số dòng (thay IF COUNT(*) FROM INSERTED)

```
-- Tạo package để lưu biến đếm dùng chung
CREATE OR REPLACE PACKAGE pkg_state AS
    g_row_count NUMBER := 0;
END pkg_state;
/

CREATE OR REPLACE TRIGGER trg_compound_vi_du
FOR UPDATE ON ten_bang
COMPOUND TRIGGER

    -- Chạy trước toàn bộ câu lệnh → reset đếm
    BEFORE STATEMENT IS
    BEGIN
        pkg_state.g_row_count := 0;
    END BEFORE STATEMENT;

    -- Chạy cho từng dòng → đếm và kiểm tra
    BEFORE EACH ROW IS
    BEGIN
        pkg_state.g_row_count := pkg_state.g_row_count + 1;
        IF pkg_state.g_row_count > 1 THEN
            RAISE_APPLICATION_ERROR(-20001, 'Chỉ được cập nhật 1 dòng');
        END IF;
    END BEFORE EACH ROW;

END trg_compound_vi_du;
/
```

6. Trigger DELETE – Hoàn trả giá trị

Dùng :OLD để lấy giá trị bị xóa

```
CREATE OR REPLACE TRIGGER trg_sau_xoa
AFTER DELETE ON hoadon
FOR EACH ROW
BEGIN
    -- Cộng lại số lượng khi hóa đơn bị xóa
    UPDATE hang
    SET soluong = soluong + :OLD.soluongban
    WHERE mahang = :OLD.mahang;
END trg_sau_xoa;
/
```

7. Trigger UPDATE – Tính chênh lệch

Dùng :NEW và :OLD để tính chênh lệch khi UPDATE

```
CREATE OR REPLACE TRIGGER trg_truoc_update
BEFORE UPDATE ON hoadon
FOR EACH ROW
BEGIN
    -- Điều chỉnh tồn kho theo chênh lệch
    UPDATE hang
    SET soluong = soluong - (:NEW.soluongban - :OLD.soluongban)
    WHERE mahang = :NEW.mahang;
END trg_truoc_update;
/
```

8. Hàm (FUNCTION) trong Oracle

CREATE OR REPLACE FUNCTION

```
CREATE OR REPLACE FUNCTION fn_TongTien (
    p_tenhang IN VARCHAR2
) RETURN NUMBER AS
    v_tong NUMBER := 0;
BEGIN
    SELECT SUM(soluong * giaban) INTO v_tong
    FROM nhatkysanhang nk
    JOIN mathang mh ON nk.mahang = mh.mahang
    WHERE mh.tenhang = p_tenhang;
    RETURN NVL(v_tong, 0);
END fn_TongTien;
/

-- Gọi hàm:
SELECT fn_TongTien('Nokia') FROM DUAL;
```

PHIẾU BÀI TẬP 1

Cấu trúc CSDL:

Hang(Mahang, Tenhang, Soluong, Giaban)

Hoadon(Mahd, Mahang, Soluongban, Ngayban)

Bài 1 – Trigger INSERT trên bảng HOADON

Tạo trigger kích hoạt khi INSERT vào bảng HOADON. Kiểm tra:

- Mahang có tồn tại trong bảng HANG không? Nếu không → đưa ra thông báo lỗi và hủy giao dịch.
- Soluongban $\leq$ Soluong tồn kho? Nếu không → thông báo lỗi và hủy giao dịch.
- Nếu thỏa mãn → UPDATE: $Soluong = Soluong - Soluongban$.

-- Gọi trigger (test):

Bài 2 – Trigger DELETE trên bảng HOADON

Tạo trigger kích hoạt khi DELETE trên bảng HOADON. Cập nhật lại bảng HANG:

- $Soluong = Soluong + Soluongban$ (của dòng bị xóa)

-- Gọi trigger (test):

Bài 3 – Trigger UPDATE trên bảng HOADON

Tạo trigger kích hoạt khi UPDATE trên bảng HOADON. Cập nhật lại bảng HANG:

- $Soluong = Soluong - (SoluongbanMoi - SoluongbanCu)$

-- Gọi trigger (test):

PHIẾU BÀI TẬP 2

Cấu trúc CSDL QL BH:

Mathang(Mahang, Tenhang, Soluong)

Nhatkybanhang(Stt, Ngay, Nguoimua, Mahang, Soluong, Giaban)

Tạo bảng và dữ liệu mẫu (Oracle)

```
CREATE TABLE Mathang (  
    Mahang    VARCHAR2(5)    CONSTRAINT pk_mathang PRIMARY KEY,  
    Tenhang   VARCHAR2(50)   NOT NULL,  
    Soluong   NUMBER(10)  
);  
  
CREATE TABLE Nhatkybanhang (  
    Stt       NUMBER          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    Ngay      DATE,  
    Nguoimua  VARCHAR2(50),  
    Mahang    VARCHAR2(5)     REFERENCES Mathang(Mahang),  
    Soluong   NUMBER(10),  
    Giaban    NUMBER(15,2)  
);  
  
-- Dữ liệu mẫu Mathang  
INSERT INTO Mathang VALUES ('1', 'Hang A', 100);  
INSERT INTO Mathang VALUES ('2', 'Hang B', 200);  
INSERT INTO Mathang VALUES ('3', 'Hang C', 150);  
COMMIT;
```

a. Trigger trg_nhatkybanhang_insert

Trigger kích hoạt khi INSERT vào bảng NHATKYBANHANG. Tự động giảm số lượng trong bảng MATHANG: $Soluong = Soluong - :NEW.Soluong$.

-- Test:

b. Trigger trg_nhatkybanhang_update_soluong

Trigger kích hoạt khi UPDATE cột SOLUONG trên bảng NHATKYBANHANG. Cập nhật lại MATHANG: $Soluong = Soluong - (SoluongMoi - SoluongCu)$.

-- Test:

c. Trigger INSERT có kiểm tra số lượng hợp lệ

Trigger BEFORE INSERT trên NHATKYBANHANG. Kiểm tra: nếu số lượng bán > số lượng tồn kho → hủy thao tác. Ngược lại → giảm số lượng trong MATHANG.

-- Test:

d. Trigger UPDATE kiểm soát số dòng (Compound Trigger)

Trigger UPDATE trên NHATKYBANHANG: nếu câu lệnh UPDATE tác động lên nhiều hơn 1 bản ghi → thông báo lỗi và hủy giao dịch. Ngược lại → cập nhật số lượng trong MATHANG.

(Gợi ý: Dùng Compound Trigger + Package biến đếm)

-- Test:

e. Trigger DELETE kiểm soát số dòng

Trigger DELETE trên NHATKYBANHANG: nếu câu lệnh DELETE tác động lên nhiều hơn 1 bản ghi → thông báo lỗi và hủy giao dịch. Ngược lại → cộng lại số lượng vào MATHANG.

-- Test:

f. Trigger UPDATE nâng cao – Kiểm tra nhiều điều kiện

Trigger UPDATE trên NHATKYBANHANG: nếu UPDATE > 1 bản ghi → lỗi. Nếu số lượng cập nhật < số lượng tồn kho → lỗi sai cập nhật. Nếu số lượng = tồn kho → thông báo không cần cập nhật. Ngược lại → cập nhật giá trị.

g. Thủ tục xóa MATHANG (có tác động 2 bảng)

Viết thủ tục nhận tham số Mahang từ bàn phím. Kiểm tra mahang có tồn tại không. Nếu không → thông báo. Nếu có → xóa dữ liệu liên quan trong NHATKYBANHANG trước, sau đó xóa MATHANG.

h. Hàm tính tổng tiền theo tên hàng

Viết hàm nhận tham số tên hàng. Tính tổng tiền = SUM(Soluong × Giaban) cho tất cả các lần bán của mặt hàng đó trong bảng NHATKYBANHANG.

-- Test:

PHIẾU BÀI TẬP 3

Cấu trúc CSDL:

Lophoc(Malop, Tenlop, Sisotoida, Sisohientai)

Dangky(Madk, Masv, Malop, Ngaydangky)

Bài 1 – Trigger INSERT trên bảng DANGKY

Tạo trigger kích hoạt khi INSERT vào bảng DANGKY. Kiểm tra:

- Malop có tồn tại trong bảng LOPHOC không? Nếu không → đưa ra thông báo lỗi và hủy giao dịch.
- Sinh viên đã đăng ký lớp đó chưa? Nếu đã tồn tại cặp Masv, Malop trong bảng DANGKY → thông báo lỗi và hủy giao dịch.
- Sisohientai < Sisotoida? Nếu lớp đã đầy → thông báo lỗi và hủy giao dịch.
- Nếu thỏa mãn → UPDATE bảng LOPHOC:
 $Sisohientai = Sisohientai + 1$

-- Gọi trigger (test):

Bài 2 – Trigger DELETE trên bảng DANGKY

Tạo trigger kích hoạt khi DELETE trên bảng DANGKY. Cập nhật lại bảng LOPHOC:

- Khi xóa một dòng đăng ký, giảm sĩ số hiện tại của lớp tương ứng:
 $Sisohientai = Sisohientai - 1$

-- Gọi trigger (test):

Bài 3 – Trigger UPDATE trên bảng DANGKY

Tạo trigger kích hoạt khi UPDATE trên bảng DANGKY. Cập nhật lại bảng LOPHOC trong trường hợp sinh viên chuyển từ lớp cũ sang lớp mới:

- Nếu MalopMoi không tồn tại trong bảng LOPHOC → thông báo lỗi và hủy giao dịch.
- Nếu sinh viên đã tồn tại trong lớp mới → thông báo lỗi và hủy giao dịch.
- Nếu lớp mới đã đầy ($Sisohientai \geq Sisotoida$) → thông báo lỗi và hủy giao dịch.
- Nếu thỏa mãn:
 - Giảm sĩ số lớp cũ: $Sisohientai = Sisohientai - 1$
 - Tăng sĩ số lớp mới: $Sisohientai = Sisohientai + 1$

-- Gọi trigger (test):